JavaScript-03 第11回_まとめ

第11回の目標

- **ScrollTriggerの導入方法**について知る
- **ScrollTriggerの活用事例**を知る
- **連続アニメーションの作成の仕方**について知る

ScrollTriggerとは

GSAPのプラグインの1つで、「スクロールして特定の位置に来たら」アニメーションを発火させることができる。
- 📎 スクロールトリガーが使われている実際のサイト

GSAP Pluginsとは

GSAPの機能を拡張するもの。要素をドラッグ可能にする、SVGのパスに沿って動かすなど、自由度の高いUIアニメーションを作れる（一部は有料会員限定）。ScrollTriggerはその1つで、**GSAPが読み込まれている環境でのみ機能する**。

ScrollTriggerでできること

1. 簡潔なコードでスクロールアニメーションを作れる
2. 「マーカー機能」で、アニメーションの開始・終了位置に簡単に目印をつけられる
3. 処理が最適化されているので、Webサイトが重くなりにくい

JavaScriptやjQueryで自作することもできるが、複雑になるほど大変で、最適化しないと動作が重くなる。予期しない挙動が起きたときに原因を探すのも大変になる。

導入

GSAP → ScrollTriggerの順で、headタグ内に読み込む（使用するHTMLファイルごとに確認する）。

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.11.3/gsap.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.11.3/ScrollTrigger.min.js"></script>
<script src="js/main.js" defer></script>
```

- ScrollTriggerはGSAPの機能を使うため、必ずGSAPより後に読み込む

- CDNのコードは、GSAP公式サイトのインストーラー（Installation）でScrollTriggerにチェックを入れると、「Browse Code」のGSAPのコードの下に表示される。他のプラグインも同じ方法で導入できる
- GSAPの導入方法／GSAP Pluginsの導入

GSAPの復習（Tween・Timeline）

ScrollTriggerは、Tween（`gsap.to()`など）やTimelineの**プロパティの1つ**として書くので、まずGSAPの基本を押さえておく（第10回参照）。

Tweenの書き方

```
// to()：今の状態 → 指定した値へ
gsap.to(".box", { rotation: 27, x: 100, duration: 1 });

// from()：指定した値 → 今の状態へ
gsap.from(".class", { opacity: 0, y: 100, duration: 1 });

// fromTo()：変化前と変化後の両方を指定
gsap.fromTo(".box",
  { x: 120, opacity: 0, rotation: 0 },              // 変化前
  { x: 0, opacity: 1, rotation: 360, duration: 4 }  // 変化後
);
```

GSAPのプロパティとCSSの対応（CSSPlugin）

GSAP	CSS
`x: 100` / `y: 100`	`transform: translateX(100px)` / `translateY(100px)`
`xPercent: -50` / `yPercent: -50`	`transform: translateX(-50%)` / `translateY(-50%)`
`rotation: 360`	`transform: rotate(360deg)`
`rotationX: 360` / `rotationY: 360`	`transform: rotateX(360deg)` / `rotateY(360deg)`
`scale: 2`	`transform: scale(2, 2)`
`scaleX: 2` / `scaleY: 2`	`transform: scaleX(2)` / `scaleY(2)`
`skewX: 45` / `skewY: 45`	`transform: skewX(45deg)` / `skewY(45deg)`

- 単位をつけない数値は、`x`/`y`ならpx、`rotation`ならdegとして扱われる
- `%`で動かしたいときは`xPercent`/`yPercent`を使う（第5回の`translate(-50%, -50%)`と同じ）

スペシャルプロパティ

プロパティ	意味
`delay`	アニメーションが開始するまでの遅延時間
`onComplete`	アニメーションの終了時に呼び出される関数
`onUpdate`	アニメーションが更新されるたびに呼び出される関数
`ease`	変化の緩急（後述の「イージング」参照）
`stagger`	複数の要素の開始時間をずらす（後述）

Timelineを使うメリット

```
// Timelineなし：前のアニメーションの時間ぶん、delayを計算して足していく必要がある
gsap.to("#id", { x: 100, duration: 1 });
gsap.to("#id", { y: 50, duration: 1, delay: 1 });
gsap.to("#id", { opacity: 0, duration: 1, delay: 2 });

// Timelineあり：つなげるだけで順番に実行される
gsap.timeline()
  .to("#id", { x: 100, duration: 1 })
  .to("#id", { y: 50, duration: 1 })
  .to("#id", { opacity: 0, duration: 1 });

// リピートさせる（2回繰り返し、間に1秒あける）
const tl = gsap.timeline({ repeat: 2, repeatDelay: 1 });
tl.to("#id", { x: 100, duration: 1 });
tl.to("#id", { y: 50, duration: 1 });
```

- 並び替えや、1か所だけアニメーション時間を変えるのも簡単になる
- 🖱️ GSAPを使ってアニメーションをつけよう

ScrollTriggerの基本の書き方

```
// シンプルな書き方：.boxが画面内に入ったらアニメーション開始
gsap.to(".box", {
  scrollTrigger: ".box",
  x: 500,
```

```
});

// 詳しく設定する書き方：オブジェクトで複数の設定をする
gsap.from(".box", {
  scrollTrigger: {
    trigger: ".trigger",
    start: "top center",
  },
  y: 100,
});

// Timeline全体にScrollTriggerをつけることもできる
const tl = gsap.timeline({
  scrollTrigger: {
    // 設定
  },
});
```

scrollTriggerに設定する主なプロパティ

プロパティ	値（初期値）	効果
`trigger`	セレクタ名	開始位置を計算する目印（トリガー）の要素
`start`	`"top bottom"`	開始位置。「要素の中での位置 画面（ビューポート）内での位置」の順で指定
`end`	`"bottom top"`	終了位置。指定の順番は`start`と同じ
`markers`	`false`	`true`で開始・終了位置を画面に表示する（確認用。本番では消す）
`scrub`	`false`	`true`でスクロール量にアニメーションの進み具合を連動させる

- `start`の初期値`"top bottom"`は「要素の上端が画面の下端に来たら」＝要素が画面に入ってきた瞬間
- 位置は`top`／`center`／`bottom`のほか、`"top 80%"`のように%やpxでも指定できる

実装例：スクロールに連動して動かす（scrub）

```
const tl = gsap.timeline({
  scrollTrigger: {
    trigger: ".trigger",
    start: "center bottom", // .triggerの中央が画面下部に来たら開始
    end: "center top",      // .triggerの中央が画面上部に来たら終了
```

```
    scrub: true,            // スクロールに合わせて進む（戻すと逆再生）
    markers: true,
  },
});

tl.to(".box", { yPercent: 350, duration: 1 });
tl.to(".box", { rotation: 360, duration: 3 });
tl.to(".box", { xPercent: 350, duration: 1 });
```

- `scrub`なし：開始位置を過ぎたら、アニメーションが最後まで自動で再生される
- `scrub: true`：スクロールした分だけアニメーションが進み、上にスクロールすると巻き戻る
- `start`〜`end`の間で、Timelineの3つの動きが順番に進む
- ScrollTriggerはプロパティが多いが、全部覚える必要はない。困ったら公式ドキュメントを見る（検索するときは使っているバージョンに注意）
- GSAPでスクロールアニメーションを実装しよう

演習1：スクロールで順番に表示する（problemセクション）

仕様

- 各パネル（`.js_problem`）をスクロールで順番に表示させる
- 1秒かけて、200px下から、見えない状態から表示させる
- 緩急の付け方は`Power4.inOut`
- セクションタイトルの上辺が画面の中央に来たときに発火（目印は`.js_problem-trigger`）

```
gsap.from(".js_problem", {
  y: 200,
  autoAlpha: 0,
  duration: 1,
  ease: "Power4.inOut",
  scrollTrigger: {
    trigger: ".js_problem-trigger", // 目印の要素
    start: "top center",            // 目印の上端が画面の中央に来たら開始
    // markers: true,               // 発火位置を画面にマーカー表示する（確認用）
  },
  stagger: {
    each: 0.6,     // 0.6秒ずつずらして次の要素を動かす
    from: "start", // 最初の要素から順番に動かす
```

```
  },
});
```

- `scrollTrigger`の中に、どこで発火させるかを書く
  - `trigger`：発火の目印にする要素
  - `start: "top center"`：2つの値が「要素のどこ」が「画面のどこ」に来たら、という指定（ここでは「要素の上端が画面の中央」）
  - `markers: true`：発火位置・終了位置を画面に表示できる。位置調整のときに便利
- `ease`：動きの緩急（加速・減速）をつける。 Easingの種類（GSAP公式）
- `stagger`：同じクラスの要素が複数あるとき、時間差をつけて順番に動かす
  - `each`：1つずつずらす秒数
  - `amount`：全体でかける秒数（`each`と`amount`はどちらか一方を指定する）
  - `from`：どの要素から順番に動かすか（`"start"`＝最初から）
- Staggerを使う

eachとamountの違い

```
gsap.from(".box", {
  scrollTrigger: {
    trigger: ".trigger",
    start: "center bottom",
    end: "center top",
  },
  duration: 1.2,
  y: 15,        // 少し下から
  opacity: 0,
  ease: "power2.out",
  stagger: {
    from: "start", // 左側（最初の要素）から
    amount: 0.8,   // 全体のずらし時間を0.8秒にする
  },
});
```

- `each: 0.6`：要素が何個あっても、**1つずつ0.6秒**ずらす → 要素が増えるほど全体の時間が長くなる
- `amount: 0.8`：**全体で0.8秒**の中に収まるようにずらす → 要素が増えるほど1つあたりの間隔は短くなる
- 個数が決まっているなら`each`、個数が変わっても全体の長さを揃えたいなら`amount`が向いている

【発展】GSAPを使わずに同じ動きをつける（Intersection Observer）

```
const problemTrigger = document.querySelector(".js_problem-trigger");
const problems = document.querySelectorAll(".js_problem");

const observer = new IntersectionObserver(
  (entries) => {
    entries.forEach((entry) => {
      // 目印が画面に入ったら
      if (entry.isIntersecting) {
        problems.forEach((problem, index) => {
          // 0.6秒ずつずらしてクラスを追加
          setTimeout(() => {
            problem.classList.add("is-active");
          }, index * 600);
        });
        observer.unobserve(problemTrigger); // 一度動いたら監視をやめる
      }
    });
  },
  {
    threshold: 0.5, // 目印が50%見えたら
  }
);

observer.observe(problemTrigger);
```

```
.js_problem {
  opacity: 0;
  visibility: hidden;
  transform: translateY(200px);
  transition: 1s ease;
}

.js_problem.is-active {
  opacity: 1;
  visibility: visible;
  transform: translateY(0);
}
```

- `IntersectionObserver`：要素が画面に入ったかどうかを監視できる、JavaScript標準の仕組み
- `setTimeout(処理, ミリ秒)`：指定した時間が経ってから処理を実行する。`index * 600`で、要素ごとに0.6秒ずつ遅らせている
- [Intersection Observer]「スクロールして特定の位置をすぎたらふわっと表示」を簡単に実装する方法

イージング（ease）

アニメーションの変化の緩急（スピードの変化）を決めるもの。モーションデザインで最も重要な部分とも言われ、選び方でアニメーションに個性が出る。

```
gsap.to(".green", { rotation: 360, duration: 2, ease: "none" });        // 一定の速さで回る
gsap.to(".purple", { rotation: 360, duration: 2, ease: "bounce.out" }); // 加速して、最後に跳ねて止まる
```

3つのタイプ（in / out / inOut）

書き方	動き	イメージ
`"power1.in"`	ゆっくり始まり、速く終わる	重い物が落ちる
`"power1.out"`	速く始まり、ゆっくり終わる	転がるボールが止まる
`"power1.inOut"`	ゆっくり始まり、ゆっくり終わる	車が加速して減速する

- UIのアニメーションには`out`系（`"power1.out"`など）が最適。速く始まるので操作への反応が良く感じられ、最後はゆっくり止まるので自然に見える
- 授業では`"Power4.inOut"`と書いているが、公式ドキュメントでの現在の書き方は小文字の`"power4.inOut"`

主なイージングの種類

名前	特徴
`none`	一定の速さ（緩急なし）
`power1`〜`power4`	数字が大きいほど緩急が強くなる（`power1`＝quad、`power2`＝cubic、`power3`＝quart、`power4`＝quint）
`sine` / `circ` / `expo`	なめらか〜急激な緩急
`back`	一度少し行き過ぎてから戻る
`bounce`	ボールのように跳ねる
`elastic`	ゴムのように伸び縮みする

名前	特徴
`steps`	段階的にカクカク動く

- 公式サイトのイーズビジュアライザーで、名前をクリックすると動きをグラフで確認できる
- `slow`・`rough`・`expoScale`や、自作の`CustomEase`などはGSAP本体に含まれていないので、別途読み込みが必要
- Easing（GSAP公式）

演習2：連続アニメーション（infoセクション）

仕様

- 各セクションをそれぞれアニメーションさせる
- 1つ目（奇数番目）は左から、2つ目（偶数番目）は右から
- 見えない状態から表示させる。緩急は`Power4.inOut`
- 各セクションの上辺が画面中央に来たときに発火
- アニメーションさせる要素は`.js_h-slide`

```
const items = document.querySelectorAll(".js_h-slide");


items.forEach((item, idx) => {
  gsap.from(item, {
    // インデックス番号に+1して2で割った余りが0（偶数番目）なら右から、そうでなければ左から
    x: (idx + 1) % 2 == 0 ? "100%" : "-100%",
    autoAlpha: 0,
    ease: "Power4.inOut",
    duration: 1,
    scrollTrigger: {
      trigger: item.parentNode, // itemの親要素をトリガーにする
      start: "top center",
    },
  });
});
```

- 1つずつ別のタイミングで発火させたいので、`forEach`で要素ごとに`gsap.from()`を設定している（`stagger`は1つの目印でまとめて動かすとき）
- 同じ動きを別の要素にもつけたいときは、その要素にも`.js_h-slide`をつけるだけでよい。同じクラス名を使い回すと、JSの記述を増やさずに済む（チャレンジ問題の答え）

- `forEach((値, インデックス番号) => {})`：インデックス番号は0から始まるので、「何番目か」にするには`+1`する
- 📎 forEach（ループ処理）／参考サイト（for・forEachでScrollTrigger）

三項演算子（if文を1行で書く）

```
// 条件 ? 条件がtrueのときの値 : 条件がfalseのときの値
x: (idx + 1) % 2 == 0 ? "100%" : "-100%"
```

上と同じことをif文で書くと次のようになる。

```
x: function () {
  if ((idx + 1) % 2 == 0) {
    return "100%";
  } else {
    return "-100%";
  }
},
```

- 📎 条件（三項）演算子（MDN）

親要素を動的に取得する

- `item.parentNode`：その要素の**すぐ上の親要素**を取得する
- `item.closest(".js_h-slide_trigger")`：その要素から上にさかのぼって、**指定したセレクタに一致する最初の祖先要素**を取得する。HTMLの階層が変わっても目印のクラスで探せるので壊れにくい

parentNodeとparentElementの違い

プロパティ	返すもの
`parentNode`	自分を包んでいるものなら何でも（`document`も含む）
`parentElement`	自分を包んでいる**HTMLの要素（タグ）**だけ。要素でなければ`null`

- 箱のたとえ：お部屋（`document`）の中に大きな箱（`<html>`）、その中に小さな箱（`<div>`など）、その中におもちゃがある
  - おもちゃに「何の中にいる？」と聞くと、どちらも「小さな箱」と答える
  - 大きな箱に聞くと、`parentNode`は「お部屋（`document`）」、`parentElement`は「箱には入っていない（`null`）」と答える
- ほとんどの場面では同じ結果になり、違うのは一番外側の`<html>`タグのときだけ

【＋α】サイトに使われている技術を調べる

- Wappalyzer（Chrome拡張機能）：見ているサイトがどんな技術で作られているかを一覧で表示してくれる
- 気になるサイトでGSAPなどが使われているか確認してみると面白い
- parentNode（MDN）／parentNodeの使い方（Qiita）

予習・復習ポイントの補足

staggerのfrom

値	動く順番
`from: "start"`	最初の要素から順番に
`from: "end"`	最後の要素から逆順に

- staggerには他にもオプションがある（staggerで順番にアニメーションさせる方法／staggerを使ったアニメーション）

forEachで要素ごとにスタイルを変える

```
const colorItem = document.querySelectorAll(".foreach_item");

// item：今処理している要素、idx：何番目か（0から始まる。indexの略）
colorItem.forEach(function (item, idx) {
  // 偶数番目は赤、奇数番目は青にする
  item.style.backgroundColor = (idx + 1) % 2 == 0 ? "red" : "blue";
});
```

- `querySelectorAll`は、配列のようなリスト（NodeList）を返す
- 引数名（`item`・`idx`）は自由だが、何を表すか分かる名前にする

AIの使い方の注意

- 模範解答をAIに渡して解説してもらい、分からない部分は「〇〇の部分が理解できないので、より詳しく基礎から教えて」と追加で聞くと深掘りできる
- AIは嘘をつくこともある。回答を参考にしつつ、自分でも検索して正しいか確認する

参考になる記事

- ScrollTrigger入門／ScrollTriggerの使い方とサンプル
- ScrollTriggerのプロパティ一覧
- GSAP easingの使い方／Easing（GSAP公式Docs）
- GSAPの基本的な記述まとめ／GSAPの全プラグイン紹介
- コンソールの表示とエラーの見方
- 一つの記事で理解が深まらなければ「〇〇 入門」などで検索し、他の記事も読む

本日のまとめ

1. ScrollTriggerはGSAPの後に読み込み、`scrollTrigger: { trigger, start }`で発火位置を指定する
2. `start: "top center"`は「要素のどこ」が「画面のどこ」に来たら、という指定
3. `stagger`を使うと、複数の要素を時間差で順番に表示できる
4. 要素ごとに別々に発火させたいときは、`forEach`で1つずつアニメーションを設定する
5. 三項演算子でif文を1行で書け、`parentNode`／`closest()`で親要素を動的に取得できる
6. `ease`は`in`／`out`／`inOut`の3タイプ。UIには`out`系が自然に見える
7. `scrub: true`でスクロール量にアニメーションを連動させられる

参考記事

- GSAPでスクロールアニメーションを実装しよう
- GSAPでスクロールアニメーションを実装しよう（別URL・同内容）
- GSAPを使ってアニメーションをつけよう（Tweenについて）
- ScrollTriggerの使い方
- Staggerを使う
- Easingの種類（GSAP公式）
- JavaScriptのループ処理を学ぼう（forEach）
- [Intersection Observer]「スクロールして特定の位置をすぎたらふわっと表示」を簡単に実装する方法
- GSAP ScrollTriggerをforやforEachで使う（参考サイト）
- 条件（三項）演算子（MDN）
- parentNode（MDN）
- parentNodeの使い方（Qiita）
- スクロールトリガーが使われている実際のサイト